Comparison Table

Research conducted using: Multiple AI Models | Run ID: 001 | Date: 02/01/26

Table of Contents

Comparison Table 2
Project Health Assessments 5
Top 5 Recommendations 6
Up-and-Coming / Emerging 8

Comparison Table

Project	Dynamic Discovery	Web UI	Docker	Streamable HTTP	Local⇄Remote Bridging	Profiles/ Namespaces	Tool Curation	API Key Mgmt	Registry/Directory	Context Injection	Health & Activity
MCP Gateway & Registry (agentic-community) (agentic-community.github.io)	True; semantic vector-based tool matching via FAISS and sentence embeddings (agentic-community.github.io)	Yes (Admin UI)	Yes (Docker/ K8s)	Yes (Streamable HTTP)	Yes; can proxy local stdio servers via HTTP/ Streamable and forward to remote services (microsoft.github.io) (microsoft.github.io)	Yes; supports grouping e.g. bundles/ Kits (enterprise features)	Yes; admins can whitelist/ expose specific tools	Yes (OAuth integration)	Yes; central MCP server registry built-in	Limited (focus is not on injection beyond tools)	Active community (≈60 forks, ongoing commits; Keycloak, Cognito integration) (github.com)
IBM Context Forge (ContextForge) (github.com)	Partial; no built-in semantic tool search. Provides MCP/ REST/A2A federation and virtual servers (github.com)	Yes (Admin UI)	Yes (Docker, PyPI)	Yes (Streamable HTTP)	Yes; can combine/federate MSP/A2A, supports multi-transport bridging and REST-to-MCP adapters (github.com)	Yes; built-in multi-tenant, RBAC (enterprise)	Yes; virtualizes REST APIs as MCP Tools	Yes (OAuth, tokens)	Yes; federated gateway with caching	Limited (security/policies focused)	Strong backing (IBM-led, 400+ forks, active releases; production-ready) (github.com) (github.com)
Unla (AmoyLab) (github.com) (github.com)	No; exposes all converted tools by default (no semantic filtering)	Yes (Web UI)	Yes (Docker)	Yes (Streamable HTTP)	Yes; proxies existing MCP servers and APIs (Client→Gateway→Tool) (github.com)	No (basic UI per server)	Yes; admins add/enable specific servers and APIs via config	Yes; token auth for UI	No (self-hosted per user; no shared catalog)	No (focused on gatewaying only)	Active (v0.9.0 in Sep 2025; 150+ forks, active releases) (github.com) (github.com)
Gate22 (ACI.dev) (github.com)	Yes; "search" endpoint for tool lookup. Unified endpoint *discovers tools at call time* across bundles, minimizing initial context (github.com)	Yes (Admin console)	Yes (Docker)	Yes (HTTP/ SSE, Streamable)	Yes; supports remote MCP onboarding via URL. Can bundle many servers into one endpoint	Yes; bundles group servers/tools per user/team (github.com)	Yes; permission allowlists control tool exposure	Yes; org-shared or per-user credentials	Yes; acts as registry of allowed MCPs/ tools	Planned (Focus on context filtering)	Early-stage (new OSS, ~14 forks; promising but evolving)
AIRIS MCP Gateway (agiletec-inc) (github.com) (github.com)	Yes; loads minimal "top-level" tool schemas and expands on-demand via an `expandSchema` tool (github.com)	Yes (Admin/ API UI)	Yes (Docker Compose)	Yes (Streamable HTTP)	Yes; runs a unified gateway for 25+ servers, dynamically enabling/disabling tools (github.com)	Implicit (bundles per IDE/ user)	Yes; dynamic tool enable/ disable, on-demand schema expansion (github.com)	Yes; API Key / token endpoint	No (config-based tool list, not a registry)	Yes; "confidence_check" and context gating utilities (github.com)	New but active (v1.0 w/ Docker, weekly commits; small OSS community)

Notes on Requirements: All listed projects support Docker and streamable HTTP. The agentic MCP Gateway excels at *semantic* tool discovery (agentic-community.github.io), directly addressing context pollution, whereas gateways like Unla and IBM ContextForge require static filters. Local vs. remote bridging is broadly supported: Microsoft's MCP Gateway can "spin up local MCP servers behind the gateway" or forward to remote HTTP/MCP endpoints (microsoft.github.io) (microsoft.github.io), Unla similarly proxies existing MCPs (github.com), and Portkey (not in table) exclusively supports remote servers (portkey.ai). Profile/namespace management and tool restriction features vary: Gate22 and AIRIS explicitly allow bundling or gating subsets of tools (github.com) (github.com), while Unla simply exposes whatever is configured. API key/auth support and central MCP registries are strongest in the agentic and IBM offerings (OAuth integration, federated registries).

Project Health Assessments

- **MCP Gateway & Registry (agentic-community)** – Actively maintained by a broad community (GitHub, Astral/Agentic land). Dozens of contributors, regular commits and updates. Backed by Agentic community with open governance. The documentation site and GitHub show ongoing feature work (dynamic discovery, OAuth, etc.). No single-coder red flags.
- **IBM MCP Context Forge** – Corporate-backed (IBM) open-source project. Hundreds of forks and stars, a committed team. Frequent releases (v1.x series), with a published roadmap (github.com) and changelogs. Robust CI, enterprise-grade features. Actively developed and unlikely to stagnate.
- **Unla** – Rapid development track. Frequent releases (v0.7→0.9 in months (github.com)) and Docker image updates. 150+ forks and contributors in Asia (AmoyLab team). Active Discord support, issues triaged. Good activity for a small team.
- **Gate22 (ACI.dev)** – New (v0.1.x) OSS launched mid-2024. Backed by ACI.dev, small core team. Early updates (alpha features) with community Discord. Limited usage/tens of stars; architecture and UI under heavy development. Promising concept, but still in v0.
- **AIRIS MCP Gateway** – Very recent project (v1 in mid-2025). A small team (agiletec-inc) with a focused mission (code assist). Few stars/forks yet, but actively addressing a key use case (context pollution). Has working Docker deployment and public repo activity (though lines of code suggest an initial MVP). Seen demos, blog coverage. Still early-stage but functioning.

(Other notable projects: Microsoft's MCP Gateway has an official repo (microsoft/mcp-gateway) with corporate backing, though it's Kubernetes-tailored and not covered above due to local deployment preference. MetaMCP (metatool-ai) has many users but (per author) lacks semantic discovery, focusing instead on middleware and namespaces.)

Top 5 Recommendations

1. **MCP Gateway & Registry (agentic-community)** – *Best overall fit.* This open-source gateway ticks nearly every box. It offers true semantic/dynamic tool discovery ("FAISS embedding" search) (agentic-community.github.io), a user-friendly web UI, Docker/Kubernetes deployment, and full HTTP streaming support. It unifies multiple MCP servers under one endpoint with access control and flexible profiles, removing context pollution by letting the model request only needed tools on-demand via the intelligent tool-finder. It also bridges local vs. remote seamlessly (proxying stdio or HTTP MCPs) (microsoft.github.io) (microsoft.github.io). The project shows strong community health (dozens of contributors, active docs, enterprise focus). For a solo developer, it means minimal manual config – just register servers/containers and let the gateway handle routing. The only caution is familiarity: it has many features which may require some learning, but the payoff (dynamic filtering of toolsets) is worth it.
2. **AIRIS MCP Gateway (agiletec-inc)** – *Innovative context pollution solution.* Airis specifically targets the context-flood problem with on-demand schema expansion. At startup it only loads "top-level" summaries of 20+ MCP servers, then exposes an `expandSchema` tool to fetch detailed parameters when needed (github.com). This multi-phase approach achieves 90% token reduction and aligns with the user's need to avoid full tool lists. It includes a small management UI and runs standalone via Docker Compose. Airis supports session multiplexing across many MCPs, dynamic enable/disable of tools, and even a "confidence_check" utility for code tasks (github.com). While still young, the project is already tailored to solo dev runs (full Docker example, health endpoint) and has a clear roadmap. Its primary drawback is niche focus: if one needs maximum contextual security and filtering for coding assistants, Airis shines.
3. **Unla (AmoyLab)** – *Lightweight and well-maintained.* Unla.Transforms any existing MCP server or REST/gRPC/WebSocket API into an MCP-compliant endpoint with zero code changes. It provides a ready-to-run Docker image and a simple web UI for configuring endpoints (github.com) (github.com). It fully supports HTTP streaming for MCP, and can proxy both local and remote services (Client→Unla Gateway→TARGET) (github.com). This makes it excellent for bridging local tools (or legacy APIs) into your MCP ecosystem without heavy infra changes. It also includes access keys/secrets management for endpoints. The downside: Unla does *not* filter tool lists dynamically – all configured tools appear to the agent. However, its robust feature set (including API-to-MCP adapters and a nice UI) and very active development (v0.9.0 in Sep 2025) make it a reliable, quick deployment for solo work.
4. **IBM MCP Context Forge** – *Enterprise-grade registry/gateway.* While primarily targeting larger deployments, ContextForge offers a rich gateway with an admin dashboard, federated MCP+REST integration, and full HTTP/MCP bridging (github.com). It's Docker-compatible and handles authentication, rate-limiting, retries, and even virtualization of REST APIs into MCP tools out-of-the-box. Critically, it supports streamable HTTP (meeting SSE deprecation) and can act as a single entry point to many backends. The dynamic tool discovery is limited (no vector search), but it does allow composing virtual MCP servers and grouping servers into namespaces for orchestration. For a solo dev, it may be heavier than needed, but if you require strict governance and observability (with access control and multi-tenant support), it remains a solid, heavily-maintained choice.
5. **Gate22 (ACI.dev)** – *Promising OSS in alpha.* Gate22's unique "bundle" concept lets teams define *exactly which* MCP tools an agent can call, and exposes just two functions (`search` + `execute`) per bundle, dramatically cutting context load. Its description promises that even hundreds of tools can be hidden behind a **single unified endpoint**, with a built-in search to find the right one at call time (github.com). It runs as Docker and has a quick UI/CLI for server onboarding and RBAC rules. While still early (alpha release), Gate22 effectively addresses context pollution by design. For a solo dev, WatchTower chores aside, it could simplify managing multiple tool sets, albeit you'd share development risk. If you are willing to try a new project (and note it has far fewer contributors than bigger projects), Gate22's tooling (including permission allowlists and dynamic discovery) is worth monitoring and potentially adopting in future releases.

Honorable Mentions:

- **MetaMCP** – A well-used aggregator/proxy with strong namespace

support (via **MCP Bundles**) and Docker deployment. It groups MCPs under unified prefixes and supports tooling middleware, but it lacks semantic/intent-based filtering. Historically relied on static tool manifests, so it can still flood the model with all tools.

- **AgentGateway (agentgateway.dev)** – A scalable connectivity

layer (from Agentic) focusing on security/observability for agent-tool communication. It is highly active and suitable for microservices/Kubernetes, but it similarly exposes all tools unless manually pruned.

- **MCP Jungle** – A simpler-proxy+registry CLI. It centralizes MCP

server configs and uses a single MCP server for clients (via an “mcp-remote” shim). It works, but delivers full tool lists to agents (no smart discovery).

Up-and-Coming / Emerging

- **MCPHub Smart Routing** – An auto-discovery system that uses vector embeddings (pgvector) to index tools on startup and answer natural-language tool queries (docs.mcphubx.com) (docs.mcphubx.com). (Not currently an easily deployable OSS, but indicative of trend towards embedding-based tool search.)
- **Jetski (HyprMCP)** – An authentication/analytics proxy for MCP (closed-source, partially documented). Could appear as open project.
- **Promising Research:** The idea of *context-aware guards* (as per the "Lethal Trifecta" discussion) is gaining traction. Projects like **OpenEdison** (a security-focused gateway) and corporate offerings (e.g. Lunar's secure MCP gateway, Kong's LLM gateway) are likely to introduce features for selective tool exposure and context injection safeguards.
- **Agent-to-Agent (A2A) Integration:** While not directly asked about, several platforms (IBM ContextForge, agentic registry) are merging MCP tool registries with A2A agent registries. Over time, expect more hybrid gateways that manage both toolkits and multi-agent workflows.

Conclusion: For immediate solo use, the combination of semantic routing and ease-of-use points to the agentic **MCP Gateway & Registry** as #1. It best meets all "Must Have" requirements (dynamic discovery, UI, Docker, streaming, active maintenance). Alternatively, for raw context-cleanliness, the **AIRIS** gateway's delayed-schema expansion is compelling. **Unla** remains the simplest turnkey solution. Monitoring Gate22 and other semantic gateways is wise as the space evolves. Ultimately, weigh stability vs. features: older projects (IBM, MetaMCP, Unla) provide stability without semantic sweeps, while newer entrants target exactly the context problem but carry the usual early-stage caveats.

Sources: Official documentation and repos for each project (e.g. Agentic MCP Gateway & Registry (agentic-community.github.io), Microsoft MCP Gateway guide (microsoft.github.io) (microsoft.github.io), Unla README (github.com), Gate22 README (github.com), Airis README (github.com), IBM ContextForge readme (github.com)).